

UCS503P Software Engineering Project Report

Scientific Literature Intelligence Platform

A local, citation-grounded retrieval-augmented assistant for scientific papers

Submitted by:

1024030171 Paarth Ganesh
1024031154 Diya Jain

Group: [GROUP NUMBER] BE [YEAR], CSED Academic year 2026–27

Submitted to:

Ms. Paramveer Kaur (Lab Instructor)

[EVALUATION STAGE]

Computer Science and Engineering
Department

Thapar Institute of Engineering and
Technology, Patiala

Abstract

Researchers increasingly need to locate, read and compare scientific papers faster than manual reading allows, yet general-purpose language models answer questions without showing where an answer came from and may state unsupported facts. This report describes the design, implementation and verification of the *Scientific Literature Intelligence Platform*, a single-user web application that answers natural-language questions over a corpus of scientific papers and cites every claim to a specific passage and page.

Papers are imported from arXiv or uploaded as PDF files. The system extracts text page by page with PyMuPDF, splits it into 256-token chunks that keep page and character provenance, embeds the chunks with the `all-MiniLM-L6-v2` sentence encoder and stores them in PostgreSQL with the `pgvector` extension. Retrieval combines dense vector search with BM25-style full-text search through reciprocal rank fusion and re-orders the candidates with a cross-encoder. Answers are generated by a locally hosted `qwen2.5:3b` model served by Ollama; the server resolves every citation against the passages actually supplied to the model and returns an explicit “insufficient evidence” response when the evidence does not support an answer. The system also produces single-paper summaries, cross-paper syntheses, structured extractions and comparison tables. It is implemented with FastAPI, SQLAlchemy and Alembic, a React and TypeScript frontend, and Docker Compose, and is checked by a GitHub Actions pipeline.

On a version-controlled evaluation set of 50 questions over 20 natural-language-processing papers, the default retrieval configuration reached a Recall@5 of 0.875 and a mean reciprocal rank of 0.701; every citation returned in two complete question-answering runs referred to evidence supplied to the model, and nine of ten unanswerable questions were declined. The 95th-percentile question-answering latency on the reference laptop was 4.4–4.5 s, above the 3 s target. The evaluation labels were written by an AI assistant and have not yet been reviewed by a person, the retrieval configuration was selected on the same questions, and spot checks found wrong-paper answers and claims not supported by their cited passage. These limitations are reported alongside the results.

Keywords: retrieval-augmented generation, semantic search, citation grounding, local language models, scientific literature, software engineering.

Contents

Abstract	i
Abbreviations	xi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope and Exclusions	2
1.5 Intended Users	2
1.6 Contributions	2
1.7 Organisation of the Report	3
2 Literature Review and Existing Approaches	5
2.1 Paper Discovery and Metadata	5
2.2 PDF Processing and Chunking	5
2.3 Embeddings and Semantic Search	5
2.4 Vector Storage and Approximate Nearest-Neighbour Search	6
2.5 Lexical Retrieval and Rank Fusion	6
2.6 Re-ranking	6
2.7 Retrieval-Augmented Generation and Grounding	6
2.8 Evaluating RAG Systems	7
2.9 Prompt Injection	7
2.10 Local Language Models	7
2.11 Summary of Design Choices	7
3 Requirements Analysis	9
3.1 Stakeholders and User Roles	9
3.2 Functional Requirements	9
3.3 Infrastructure Requirements	11
3.4 Non-Functional Requirements	11
3.5 Use Cases	11
3.6 Assumptions and Constraints	13
3.7 Software and Hardware Requirements	14
3.8 Acceptance Criteria	14

4	System Analysis and Architecture	15
4.1	Architectural Style	15
4.2	Backend Layers and Responsibilities	15
4.3	Frontend and API Boundary	16
4.4	Data Design	16
4.5	Ingestion and Retrieval Pipelines	17
4.6	Inference and Citation Flow	17
4.7	Error Handling and External Dependencies	18
4.8	Design Decisions	18
5	UML and Detailed Software Design	21
5.1	Use Case Diagram	21
5.2	Class Diagrams	21
5.3	Sequence Diagram	25
5.4	Activity Diagram	25
5.5	Communication Diagram	25
5.6	Consistency Between the Diagrams	28
6	Implementation	29
6.1	Paper Discovery and Metadata	29
6.2	PDF Upload and Parsing	29
6.3	Chunking	29
6.4	Embedding Generation	30
6.5	Database Persistence	30
6.6	Semantic Retrieval	30
6.7	Context Assembly and Ollama Inference	31
6.8	Citation Validation and Source Navigation	31
6.9	Summaries, Synthesis, Extraction and Comparison	32
6.10	Frontend Workflows	32
6.11	Configuration and Error Handling	32
7	Testing and Evaluation	33
7.1	Test Strategy	33
7.1.1	Unit, API and Integration Tests	33
7.1.2	Database and Migration Tests	34
7.1.3	End-to-End Tests	34
7.2	Evaluation Data Set	34
7.3	Metrics	35
7.4	Retrieval Results	35
7.5	Retrieval Latency Profile	36
7.6	Question-Answering Results	36
7.7	Citation Integrity	36

7.8	Groundedness	37
7.9	Abstention, Failure Handling and Prompt Injection	38
7.10	Final Acceptance Journeys	38
7.11	Continuous Integration	38
8	Security, Privacy and Reliability	41
8.1	Implemented Controls	41
8.2	Untrusted Retrieved Text and Model Output	41
8.3	Local Model Deployment	41
8.4	Logging and Retention	41
8.5	Availability and Recovery	43
8.6	Open Risks	43
9	Results, Limitations and Future Work	45
9.1	Demonstrated Capabilities	45
9.2	Requirements Achieved	45
9.3	Known Limitations	46
9.4	Technical Debt	46
9.5	Future Work	46
9.6	Maintenance and Extension at the Institute	47
10	Conclusion	49
	References	51
A	Setup and Execution	55
A.1	Prerequisites	55
A.2	Start the Stack	55
A.3	Run the Tests	55
A.4	Run the Evaluation	56
A.5	Demonstration Sequence	56
B	API and Configuration Reference	57
B.1	REST Endpoints	57
B.2	Main Configuration Settings	58
C	Requirements Traceability Matrix	59
D	Evaluation Run Records	63
D.1	Sample Output (real)	63

List of Figures

4.1	Deployment and component architecture	15
4.2	Data flow from source document to answer citation	18
5.1	UML use case diagram	22
5.2	UML class diagram, part 1: persistent domain model	23
5.3	UML class diagram, part 2: services and abstractions	24
5.4	UML sequence diagram: grounded question answering	26
5.5	UML activity diagram: paper ingestion	27
5.6	UML communication diagram: grounded question answering	28

List of Tables

2.1	Design choices and their background	8
3.1	Stakeholders and roles	9
3.2	Functional requirements	10
3.3	Non-functional requirements	12
3.4	Use cases	13
3.5	Reference environment	14
4.1	Backend packages	16
4.2	Architecture decision records	19
5.1	Correspondence of sequence and communication messages	28
7.1	Test layers and results in the final acceptance run (9 October 2026) .	33
7.2	Retrieval quality and search latency by mode (40 answerable questions)	35
7.3	Retrieval stage latency at top 6 (ms, p50 / p95)	36
7.4	Grounded question answering over the evaluation set	37
8.1	Security and privacy controls and how they were verified	42
9.1	Requirement status at final acceptance	45
B.1	REST endpoints	57
B.2	Main configuration settings	58
C.1	Requirements traceability	59
D.1	Run records cited in this report	63

Abbreviations

Term	Meaning
ADR	Architecture Decision Record (<code>docs/adr/</code>)
ANN	Approximate nearest-neighbour (search)
API	Application Programming Interface
BM25	Okapi Best Match 25 lexical ranking function
CI	Continuous Integration (GitHub Actions)
CORS	Cross-Origin Resource Sharing
FR / IR / NFR	Functional, infrastructure and non-functional requirement identifiers
HNSW	Hierarchical Navigable Small World graph index
LLM	Large Language Model
MRR	Mean Reciprocal Rank
ORM	Object-Relational Mapping (SQLAlchemy)
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
RTM	Requirements Traceability Matrix
SSRF	Server-Side Request Forgery
UML	Unified Modeling Language

Introduction

1.1 Background and Motivation

Scientific output grows faster than any individual can read. A student or researcher who wants to know which optimiser a paper used, how two methods compare on a benchmark, or what limitations the authors acknowledge must locate the relevant papers, open each PDF and search its text by hand. Large language models (LLMs) can answer such questions fluently, but a model answering from its parameters gives no indication of where an answer came from and is known to produce plausible but unsupported statements [1]. For scientific use, an answer is only useful if the reader can check it against the source.

Retrieval-augmented generation (RAG) addresses this by first retrieving passages from a document collection and then asking the model to answer from those passages [2, 3]. Retrieval alone does not make an answer trustworthy: the model can still cite the wrong passage, cite a passage it was never given, or answer when the passages do not contain the answer. This project therefore treats *citation integrity* and *abstention* as engineering requirements that the server enforces and that the team measures, rather than as properties assumed of the model.

1.2 Problem Statement

Build a software system that lets a user ask natural-language questions over a corpus of scientific papers and returns answers in which every factual claim is traceable to a specific passage and page of a specific paper, that explicitly declines to answer when the indexed papers do not support an answer, and whose retrieval and answer quality are measured reproducibly. The corpus must be populated both from arXiv and from user-supplied PDF files.

1.3 Objectives

- O1. Ingest papers from the arXiv API and from uploaded PDF files, with validation and job tracking.
- O2. Extract text page by page and preserve provenance from paper to page, character span and chunk.

- O3. Provide semantic search over the corpus that returns ranked passages with their pages.
- O4. Answer questions with a *local* language model, cite each claim, verify every citation on the server, and abstain explicitly when evidence is insufficient.
- O5. Provide single-paper summaries, cross-paper synthesis, structured extraction and comparison.
- O6. Measure retrieval quality (Recall@ k , MRR), citation validity, abstention and latency on a version-controlled evaluation set, and gate retrieval regressions in continuous integration.
- O7. Apply software-engineering discipline throughout: requirement identifiers, traceability, architecture decision records, automated tests, security review and reproducible setup.

1.4 Scope and Exclusions

The delivered system is a single-user application intended to run on one machine with Docker Compose. In scope are the capabilities listed above together with a web interface for each of them. The following are out of scope for the minimum viable product (MVP) and are discussed in chapter 9: optical character recognition for scanned PDFs, user accounts and authorisation, public deployment, and any hosted (cloud) language model. The original proposal named a hosted model (Gemini 2.5 Flash) and a cloud deployment; the team replaced the former with local inference through Ollama and kept deployment local (decision record ADR-0001), so that no paper or question leaves the user's machine.

1.5 Intended Users

The primary users are students and researchers who maintain a working set of papers and want to query it. A secondary audience is the course evaluators and any future maintainers at the institute, for whom the repository provides requirements, decision records, test suites and operating instructions.

1.6 Contributions

The contributions of this project are engineering contributions, not new retrieval or generation methods:

- A working, documented end-to-end system combining established techniques: dense retrieval [4, 5], BM25-style lexical retrieval [6], reciprocal rank fusion [7], cross-encoder re-ranking [8] and local generation.
- A server-side citation contract: the model may only cite passage labels; labels are resolved against the passages actually supplied, and an answer without a valid citation is withheld.
- An evaluation harness with reproducible run records, a continuous-integration retrieval gate, and audits of retrieval quality, security and reproducibility whose findings are reported in this document, including the unfavourable ones.

1.7 Organisation of the Report

chapter 2 reviews the techniques the system builds on. chapter 3 states the requirements and use cases. chapter 4 presents the architecture and chapter 5 the five UML diagrams. chapter 6 describes the implementation and chapter 7 the tests and evaluation results. chapter 8 covers security, privacy and reliability, chapter 9 the results, limitations and future work, and chapter 10 concludes. Appendices give setup instructions, API and configuration references, the requirements traceability matrix and test records. The report is typeset with the TIET report class [9].

Literature Review and Existing Approaches

This chapter summarises the published techniques the platform builds on. Statements about what this project does are design choices of the team and are identified as such; they are justified by measurements in chapter 7, not by the cited works.

2.1 Paper Discovery and Metadata

arXiv exposes an Atom-based query API that returns identifiers, versions, titles, authors, abstracts, categories and links to PDF files [10]. Identifiers are versioned (for example `1706.03762v7`), so a corpus can be pinned to exact versions, which this project uses for its evaluation corpus. The API documentation asks clients to space requests; the project observed HTTP 429 responses when this was not respected and added rate limiting and retries accordingly (chapter 7).

2.2 PDF Processing and Chunking

Scientific PDFs encode text as positioned glyphs on pages. Libraries such as PyMuPDF extract text per page [11], which makes it possible to record the page on which every character lies. Because embedding models accept bounded inputs, extracted text is split into chunks. Fixed-size windows with overlap are a common baseline; the overlap reduces the chance that a sentence is split across chunk boundaries. A known weakness of plain-text extraction is that tables lose their row and column structure, which the project observed in practice (chapter 9).

2.3 Embeddings and Semantic Search

Transformer encoders [12, 13] can be fine-tuned to map sentences to vectors whose cosine similarity reflects semantic similarity. Sentence-BERT introduced siamese fine-tuning of BERT for this purpose [5]; the `all-MiniLM-L6-v2` model used here is a compact sentence encoder from that family [14, 15], based on the MiniLM distillation approach [16], and produces 384-dimensional vectors. Dense passage retrieval showed that learned dense representations can outperform lexical retrieval for open-domain question answering [4].

2.4 Vector Storage and Approximate Nearest-Neighbour Search

Exact nearest-neighbour search scans every vector. Graph-based indexes such as Hierarchical Navigable Small World (HNSW) graphs trade a small loss of recall for sub-linear query time [17]. The pgvector extension adds a vector type, distance operators and HNSW indexes to PostgreSQL [18], which lets one database hold papers, chunks, embeddings and question history with ordinary transactions and constraints. At the size of the project’s corpus the PostgreSQL planner chooses an exact scan rather than the HNSW index (chapter 7).

2.5 Lexical Retrieval and Rank Fusion

BM25 ranks documents by term frequency and inverse document frequency with length normalisation [6, 19]. PostgreSQL provides full-text search with stemming and a cover-density ranking function, `ts_rank_cd` [20]; the project uses it as a BM25-style lexical ranker, not as an exact BM25 implementation. Lexical and dense rankings fail on different queries, so combining them is attractive. Reciprocal rank fusion (RRF) scores each document by $\sum_r 1/(k + \text{rank}_r)$ over the input rankings; its authors reported that it outperformed Condorcet fusion and individual learning-to-rank methods [7].

2.6 Re-ranking

A cross-encoder reads the query and a candidate passage together and outputs a relevance score. BERT-based cross-encoders substantially improved passage ranking on MS MARCO [8]. Because the query must be re-encoded with every candidate, a cross-encoder is applied only to a short candidate list. The project uses the publicly available `cross-encoder/ms-marco-MiniLM-L-6-v2` model [21].

2.7 Retrieval-Augmented Generation and Grounding

RAG conditions a generator on retrieved passages [2]; surveys describe many variants of retrieval, context construction and evaluation [3]. Two findings from the literature shaped the design. First, models do not use long contexts uniformly and can miss information in the middle of the context [22], which motivates bounded, ranked context. Second, generated text may contain claims not supported by the input [1]. The project therefore requires the model to cite passage labels for every claim, resolves

those labels on the server, and treats a structurally valid citation as necessary but not sufficient evidence of support.

2.8 Evaluating RAG Systems

Retrieval is commonly evaluated with Recall@ k and mean reciprocal rank (MRR) [19, 23]. Answer quality is harder to measure. Reference-free frameworks such as RAGAS score faithfulness and relevance with a language model [24], and studies of “LLM-as-a-judge” report useful agreement with humans together with biases [25]. The project uses a local model as a supplementary judge only and reports its disagreements with manual inspection (chapter 7).

2.9 Prompt Injection

Text retrieved into a model’s context can contain instructions. Indirect prompt injection showed that such text can take control of LLM-integrated applications [26], and the OWASP Top 10 for LLM applications lists prompt injection first [27]. The project treats retrieved text as untrusted data: it is delimited, delimiter-like text is neutralised, the instructions forbid following it, and the output format is constrained by a JSON schema.

2.10 Local Language Models

Open-weight models can be served locally; Ollama provides a local HTTP API for running such models [28]. The project uses `qwen2.5:3b` from the Qwen2.5 family [29], chosen because it fits in the 4 GB of GPU memory on the reference laptop. Local inference keeps documents private but limits model size, which affects answer quality and latency.

2.11 Summary of Design Choices

Table 2.1 relates the main design choices of the team to the literature that motivates them.

Table 2.1: Design choices and their background

Choice in this project	Background
MiniLM sentence embeddings, cosine similarity	[5, 16, 4]
pgvector inside PostgreSQL	[18, 17]
Full-text ranking fused with dense ranking (RRF, $k = 60$)	[6, 7, 20]
Cross-encoder re-ranking of a short candidate list	[8]
Answers only from retrieved, labelled passages; server-side citation checks	[2, 1]
Bounded context (six passages by default)	[22]
Retrieved text treated as untrusted	[26, 27]
Recall@ k , MRR; local judge as a supplement only	[19, 23, 25]

Requirements Analysis

The canonical requirement register is version-controlled under `docs/requirements/`. Identifiers are permanent: functional requirements FR-01 to FR-26, infrastructure requirements IR-01 to IR-09 and non-functional requirements NFR-01 to NFR-16. This chapter summarises them and adds use-case identifiers (UC-01 to UC-13) for the UML model in chapter 5. The verification status of each requirement, determined in the final acceptance run of 9 October 2026, is given in Appendix C.

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and roles

Role	Interest and interaction
Researcher (user)	Imports and uploads papers, searches, asks questions, inspects citations, runs analyses. Single user; no accounts.
Maintainer	Runs the evaluation harness and test suites, applies migrations, reads logs and decision records.
Course evaluators	Assess requirements, design, verification evidence and documentation.
arXiv (external system)	Supplies paper metadata and PDF files over HTTPS.
Ollama (external system)	Serves the local language model used for answers and analyses.

3.2 Functional Requirements

Table 3.2 lists the functional requirements with the priority recorded in the register. “Required” requirements are mandatory for the MVP.

Table 3.2: Functional requirements

ID	Requirement	Priority
FR-01	Search the arXiv API by query or identifier and show metadata.	Required
FR-02	Import an arXiv paper; re-importing a known identifier updates rather than duplicates.	Required
FR-03	Upload a PDF with validation (type, size, pages, encryption, text layer) and safe storage.	Required
FR-04	Page-aware text extraction; every character traceable to a page and offset.	Required
FR-05	Deterministic chunking; chunks spanning pages record all pages.	Required
FR-06	Local embeddings with <code>all-MiniLM-L6-v2</code> ; dimension validated as 384.	Required
FR-07	Persist papers, documents, chunks and embeddings in PostgreSQL with pgvector, constraints and migrations.	Required
FR-08	Semantic search returning ranked passages with score, chunk, paper and pages; corpus filter.	Required
FR-09	Grounded question answering with a local Ollama model, answering only from retrieved passages.	Required
FR-10	Citations resolved on the server; unresolvable citations flagged, never shown as valid.	Required
FR-11	Explicit insufficient-evidence response when context does not support an answer.	Required
FR-12	Single-paper summary with provenance.	Required
FR-13	Cross-paper synthesis with per-claim citations.	Required
FR-14	Structured extraction (task, method, dataset, metric, result, limitations), each field cited or unknown .	Required
FR-15	Multi-paper comparison table with caveats.	Required
FR-16	Corpus browsing: list, filter, details, delete with confirmation.	Required
FR-17	Ingestion job tracking with explicit states, actionable errors and retry.	Required
FR-18	Query history with answers, citations, configuration and latency.	Required
FR-19	Retrieval evaluation harness ($\text{Recall}@k$, MRR) on a versioned labelled set.	Required
FR-20	Answer-quality evaluation: citation validity, groundness, abstention, latency.	Required

ID	Requirement	Priority
FR-21	Automated retrieval-quality gate in continuous integration.	Required
FR-22	Dashboard summarising corpus, jobs and recent questions.	Recommended
FR-23	Read-only settings view with models and dependency health.	Recommended
FR-24	Hybrid dense and lexical retrieval with rank fusion.	Recommended
FR-25	Cross-encoder re-ranking, adopted only if it improves the evaluation results.	Optional
FR-26	Optical character recognition for scanned PDFs.	Optional (deferred)

3.3 Infrastructure Requirements

IR-01 a versioned REST API (`/api/v1`) with typed schemas, a consistent error envelope and OpenAPI; IR-02 environment-based configuration with no secrets in version control; IR-03 Docker Compose for the database, backend and frontend; IR-04 continuous integration for lint, format, type checks, tests and build; IR-05 liveness and per-dependency readiness endpoints; IR-06 structured JSON logs with request identifiers and no document text; IR-07 an Ollama adapter with timeouts and error mapping; IR-08 background execution of ingestion jobs; IR-09 reproducible setup, troubleshooting and handover documentation.

3.4 Non-Functional Requirements

Non-functional requirements are targets to be measured; none is claimed without a recorded run.

3.5 Use Cases

Table 3.4 lists the use cases realised by the implementation. They are drawn in the use-case diagram (Figure 5.1).

The evaluation harness (FR-19 to FR-21) is a command-line tool used by the maintainer and in continuous integration; it is not part of the user-facing system and is therefore not drawn as a use case.

Table 3.3: Non-functional requirements

ID	Quality	Target
NFR-01	Retrieval quality	Recall@5 ≥ 0.80 on the pilot evaluation set
NFR-02	Retrieval quality	MRR reported for every retrieval configuration
NFR-03	Latency	95th-percentile end-to-end question answering ≤ 3 s (hardware, model, warm/cold stated)
NFR-04	Latency	Search latency reported separately from generation latency
NFR-05	Availability	$\geq 99\%$ availability of search and Q&A over a defined pilot window
NFR-06	Integrity	100% of displayed citations resolve to persisted chunks
NFR-07	Integrity	Abstention on unanswerable questions; precision and recall reported
NFR-08	Reproducibility	Same corpus and configuration give identical chunks and rankings
NFR-09	Reproducibility	Model, embedding model, chunk configuration, top- k and prompt version recorded
NFR-10	Security	Upload limits, path-traversal protection, parameterised SQL, CORS, secrets, prompt injection
NFR-11	Privacy	No document text or secrets in logs; no data sent to hosted LLM services
NFR-12	Resource bounds	Upload size, pages, question length, top- k , tokens, context and concurrency bounded
NFR-13	Hardware fit	Runs on the reference laptop (16 GB RAM, 4 GB GPU, Windows 11)
NFR-14	Maintainability	CI green on every pull request
NFR-15	Accessibility	Keyboard navigation, labelled controls, usable at 360 px width
NFR-16	Graceful degradation	Search works without Ollama; Q&A returns an explicit 503

Table 3.4: Use cases

ID	Use case	Actors	Requirements
UC-01	Discover papers on arXiv	Researcher, arXiv	FR-01
UC-02	Import a paper from arXiv	Researcher, arXiv	FR-02, FR-04–FR-07
UC-03	Upload a PDF	Researcher	FR-03–FR-07
UC-04	Track an ingestion job (included by UC-02, UC-03)	Researcher	FR-17
UC-05	Retry a failed job (extends UC-04)	Researcher, arXiv	FR-17
UC-06	Browse and manage the corpus	Researcher	FR-16
UC-07	Search passages	Researcher	FR-08, FR-24, FR-25
UC-08	Ask a question	Researcher, Ollama	FR-09–FR-11
UC-09	Inspect a citation (extends UC-08, UC-12)	Researcher	FR-10
UC-10	Summarise or extract a paper	Researcher, Ollama	FR-12, FR-14
UC-11	Synthesise and compare papers	Researcher, Ollama	FR-13, FR-15
UC-12	View question history	Researcher	FR-18
UC-13	View dashboard, settings and system status	Researcher	FR-22, FR-23, IR-05

3.6 Assumptions and Constraints

- One user on one machine; all application ports bind to 127.0.0.1; no authentication.
- No hosted language model, ever, as a fallback (contract rule; ADR-0001). If Ollama is unavailable, the system returns HTTP 503 with a reason.
- Python 3.12, managed by `uv` without changing the machine’s Python 3.14, because PyTorch and some scientific packages often lag new Python releases on Windows (ADR-0004).
- Text-based PDFs only; scanned PDFs are rejected.
- The evaluation labels may be written by the AI assistant used by the team (ADR-0007); every item records its labeller and review status, and reports state how many items a person reviewed.

Table 3.5: Reference environment

Item	Requirement / reference value
Hardware	AMD Ryzen 7 7435HS, 16 GB RAM, NVIDIA RTX 3050 Laptop 4 GB, Windows 11
Runtime	Docker Desktop (WSL2) with Compose; Ollama on the host with <code>qwen2.5:3b</code>
Database	PostgreSQL 17 with pgvector 0.8.0 (container image <code>pgvector/pgvector:0.8.0-pg17</code>)
Backend	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2, Alembic, PyMuPDF, sentence-transformers (CPU PyTorch)
Frontend	React, TypeScript, Vite, Vitest with Testing Library, oxlint; served by nginx 1.27
Tooling	Ruff, mypy (strict), pytest, GitHub Actions, pip-audit, npm audit

3.7 Software and Hardware Requirements

3.8 Acceptance Criteria

The register defines numbered acceptance criteria (AC-*xx.n*) for each functional requirement in `docs/requirements/acceptance-criteria.md`. The MVP was accepted on the following overall criterion: every mandatory user journey (discovery, import, upload, provenance, embedding, search, grounded answering with verified citations, abstention, analyses, corpus management, history, and graceful handling of invalid input and missing services) works against the running application with real components, and no release-blocking defect remains. The outcome (chapter 9) was *ready with documented limitations*.

System Analysis and Architecture

4.1 Architectural Style

The system is a *modular monolith*: one backend process and one single-page frontend, deployed together with the database by Docker Compose (ADR-0002). The project's engineering contract forbids microservices and plugin frameworks without a demonstrated need. A single process keeps transactions simple (a paper and all its chunks are written in one transaction) and makes the system easy to run on one laptop.

Figure 4.1 shows the deployed components. The browser talks only to nginx, which serves the built React application and proxies `/api/v1` to the FastAPI backend. The backend reaches PostgreSQL inside the Compose network, Ollama on the host through `host.docker.internal`, and arXiv over HTTPS through an allow-listed client. All published ports bind to `127.0.0.1`.

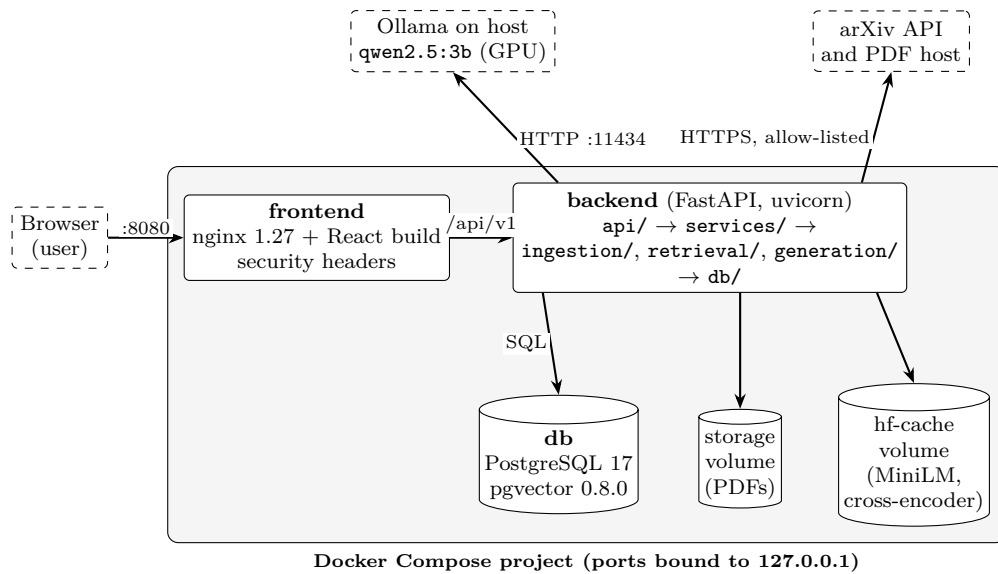


Figure 4.1: Deployment and component architecture

4.2 Backend Layers and Responsibilities

The backend is written in Python with FastAPI [30], Pydantic schemas at every boundary, and SQLAlchemy with Alembic migrations [31]; the frontend uses React

[32] with TypeScript. The backend is divided into packages with a strict dependency direction: `api` $\rightarrow$ `services` $\rightarrow$ `ingestion`, `retrieval`, `generation`, `evaluation` $\rightarrow$ `db`. Routes never contain SQL or prompts, and the generation package never touches the database.

Table 4.1: Backend packages

Package	Responsibility
<code>api/v1</code>	HTTP routes, Pydantic request and response schemas, error envelope
<code>services</code>	Use cases: ingestion jobs, search, question answering, analyses, corpus, health
<code>ingestion</code>	arXiv client, PDF validation and page-aware extraction, chunking, file storage
<code>retrieval</code>	Embedding model wrapper, dense, lexical and hybrid search, cross-encoder re-ranker
<code>generation</code>	Provider interface and Ollama adapter, prompts and output schemas, citation checking
<code>evaluation</code>	Evaluation data set, metrics, run records, provenance and profiling commands
<code>db</code>	SQLAlchemy models and sessions; schema owned by Alembic migrations
<code>core</code>	Settings, logging, typed domain errors

A composition root (`container.py`) builds the long-lived services from settings; tests build their own containers with fakes, which keeps the services independent of their concrete dependencies.

4.3 Frontend and API Boundary

The frontend is a React and TypeScript single-page application with views for the dashboard, asking questions, searching passages, the corpus, comparison, adding papers, history and settings. It calls the versioned REST API through a single typed client (`frontend/src/api/client.ts`). Every error response uses the envelope `{"error": {"code", "message"}}`, so the interface can show actionable messages. The main endpoints are listed in Appendix B.

4.4 Data Design

Nine tables hold the persistent state (`docs/architecture/data-model.md`):

- `papers`: metadata; `arxiv_id` is unique, which makes arXiv import idempotent.

- **documents:** the PDF behind a paper, with its SHA-256 (unique, for duplicate detection), storage key, page count, and the extractor, chunker and embedding-model versions used.
- **chunks:** ordinal, `page_start`, `page_end`, `char_start`, `char_end`, token count, text, a 384-dimensional **vector** with an HNSW cosine index, and a stored **tsvector** with a GIN index for lexical search.
- **ingestion_jobs:** kind, state, error, attempts; a partial unique index allows one in-flight job per source.
- **queries, answers, answer_evidence, answer_citations:** question history. Evidence rows are *snapshots* of the passages given to the model, so history survives deletion of a paper.
- **analyses:** summaries, syntheses and extractions with their evidence snapshots.

Constraints are enforced in the database (foreign keys with cascades, unique keys, and checks on page ranges, character spans and token counts). The schema changes only through Alembic migrations (0001 to 0006); a test fails if the ORM models and migrations drift apart.

4.5 Ingestion and Retrieval Pipelines

Figure 4.2 traces data from a source document to an answer citation. Ingestion validates the input, stores the PDF under a name derived from its hash, extracts text page by page while recording the character offset at which each page starts, chunks the text into 256-token windows with a 38-token overlap (ADR-0003), embeds the chunks in batches and writes everything in one transaction. Retrieval embeds the question, runs dense and lexical search, fuses the two rankings with RRF ($k = 60$) and re-orders the fused candidates with the cross-encoder (ADR-0008). The provenance chain—paper, page, character span, chunk, retrieval score, citation—is preserved end to end.

4.6 Inference and Citation Flow

Question answering keeps only passages whose cosine similarity is at least 0.30, labels them P1 to Pn, wraps them in delimiters, and sends them to Ollama together with a JSON schema that constrains the output to claims, each listing the labels it relies on, and a status (ADR-0006). The server parses the output, resolves every label against the set actually supplied, drops claims that consist only of labels, and returns “insufficient evidence” if the model abstains or if no claim has a valid cita-

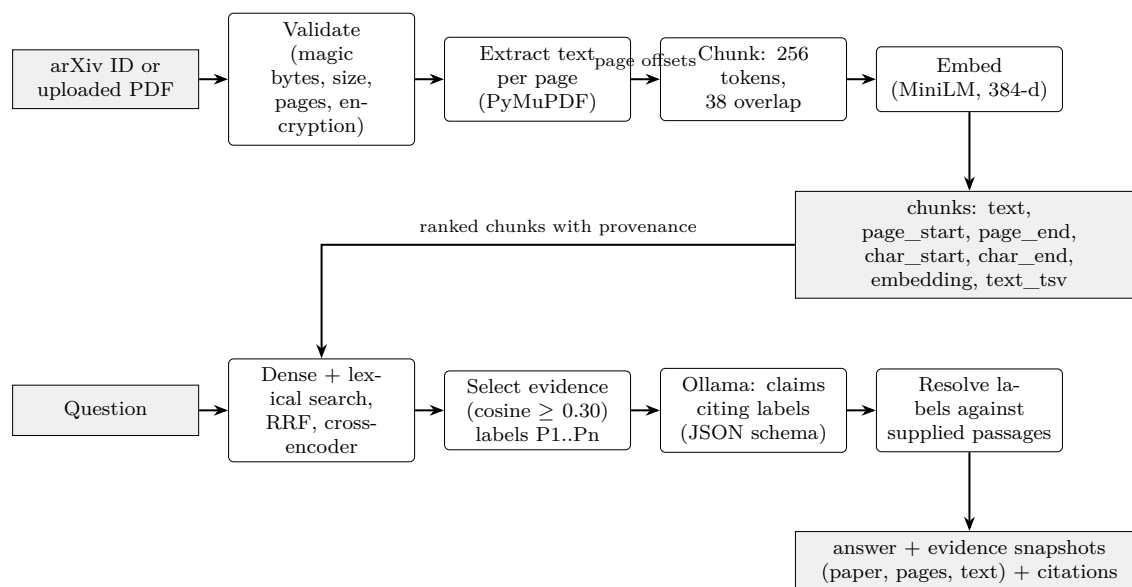


Figure 4.2: Data flow from source document to answer citation

tion. The detailed interaction is shown in the sequence and communication diagrams (Figure 5.4, Figure 5.6).

4.7 Error Handling and External Dependencies

Expected failures are typed domain errors mapped to HTTP status codes: invalid input 422, not found 404, conflict 409 (duplicate upload), upstream failure 502 (for example arXiv rate limiting), dependency unavailable 503 (Ollama or the database not reachable), and generation timeout 504. Unexpected exceptions return a fixed `internal_error` message and are logged on the server only. A readiness endpoint reports the database, embedding model, re-ranker and Ollama separately. Search keeps working when Ollama is down (NFR-16).

4.8 Design Decisions

Table 4.2: Architecture decision records

ADR	Decision	Main alternative rejected
0001	Local Ollama for generation; no hosted fallback	Hosted Gemini model named in the proposal
0002	Modular monolith backend, separate SPA, monorepo	Separate ingestion worker service
0003	Chunk window of 256 tokens with 38 overlap, within the encoder input limit	Proposal’s 500-token chunks, of which the encoder would see only about half
0004	Python 3.12 via uv; PostgreSQL with pgvector in Docker	Native Windows pgvector build
0005	In-process ingestion jobs backed by a job table	Celery with Redis
0006	Labelled evidence, constrained JSON, server-side citation checks	Prompt with a worked example (it produced a cited answer to an unanswerable question)
0007	Evaluation labels written by the AI assistant, with recorded provenance (team decision)	Labels written by the team (the contract’s original rule)
0008	Hybrid dense and lexical retrieval with cross-encoder re-ranking	Dense retrieval only

UML and Detailed Software Design

This chapter presents five UML views of the system. Each diagram was drawn from the source code named in the comment at the top of its TikZ file under `report/figures/uml/`, and the class, operation and message names are those used in the code. The diagrams are rendered by TikZ when the report is compiled, so the editable source and the figure are the same file.

5.1 Use Case Diagram

Figure 5.1 shows the system boundary, the primary actor (the researcher) and the two external systems that take part in use cases: arXiv supplies metadata and PDF files, and the local Ollama service generates answers and analyses. Importing or uploading a paper *includes* tracking its ingestion job, because both requests return a job that the interface polls. Retrying a failed job *extends* job tracking, since it happens only when a job has failed. Inspecting a citation *extends* asking a question and viewing history: it is optional behaviour available wherever an answer with citations is shown. The evaluation harness is a maintainer command-line tool and is deliberately outside the boundary (chapter 3).

5.2 Class Diagrams

The static structure is split into two diagrams to keep them legible. Figure 5.2 shows the persistent domain model from `backend/app/db/models.py`. A `Paper` owns at most one `Document` (the PDF currently behind it), which owns its `Chunks`; deleting a paper cascades to both (composition). An `IngestionJob` produces at most one paper. Question history is modelled by `Query`, `Answer`, `AnswerEvidence` and `AnswerCitation`: each citation resolves to at most one evidence row of the same answer (none if the label was invalid), and each evidence row is a snapshot of a chunk whose reference becomes empty if the chunk is later deleted. `Analysis` stores the identifiers of the papers it covers rather than a foreign key, shown as a dependency.

Figure 5.3 shows the service layer and the abstractions it depends on. The services depend on interfaces (`GenerationProvider`, `Embedder`, `RerankerProtocol`, `ArxivSource`), realised by the production classes; the tests substitute fakes for the same interfaces. Citation checking and the search queries are module-level functions, shown as «utility» classes. Only public operations are listed, and `CorpusService`'s use of `FileStore` for deleting files is drawn; framework classes are omitted.

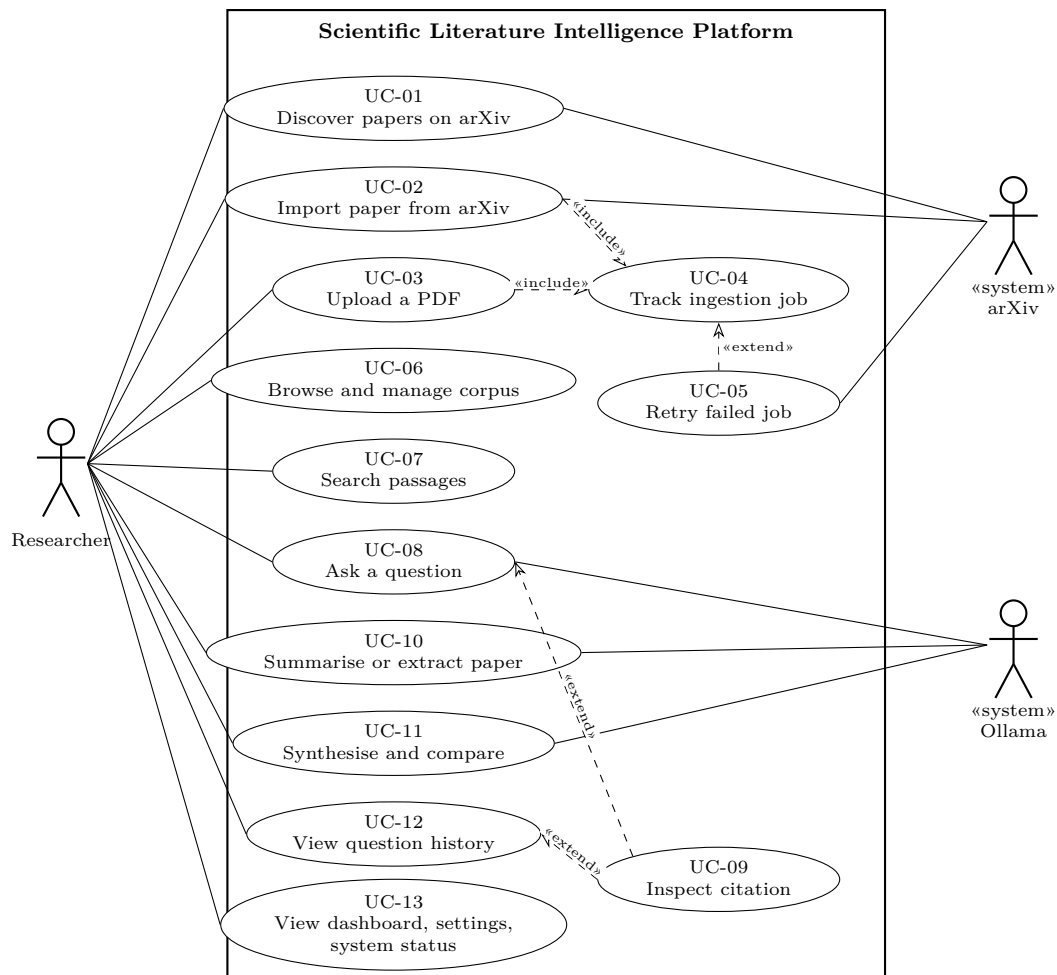


Figure 5.1: UML use case diagram

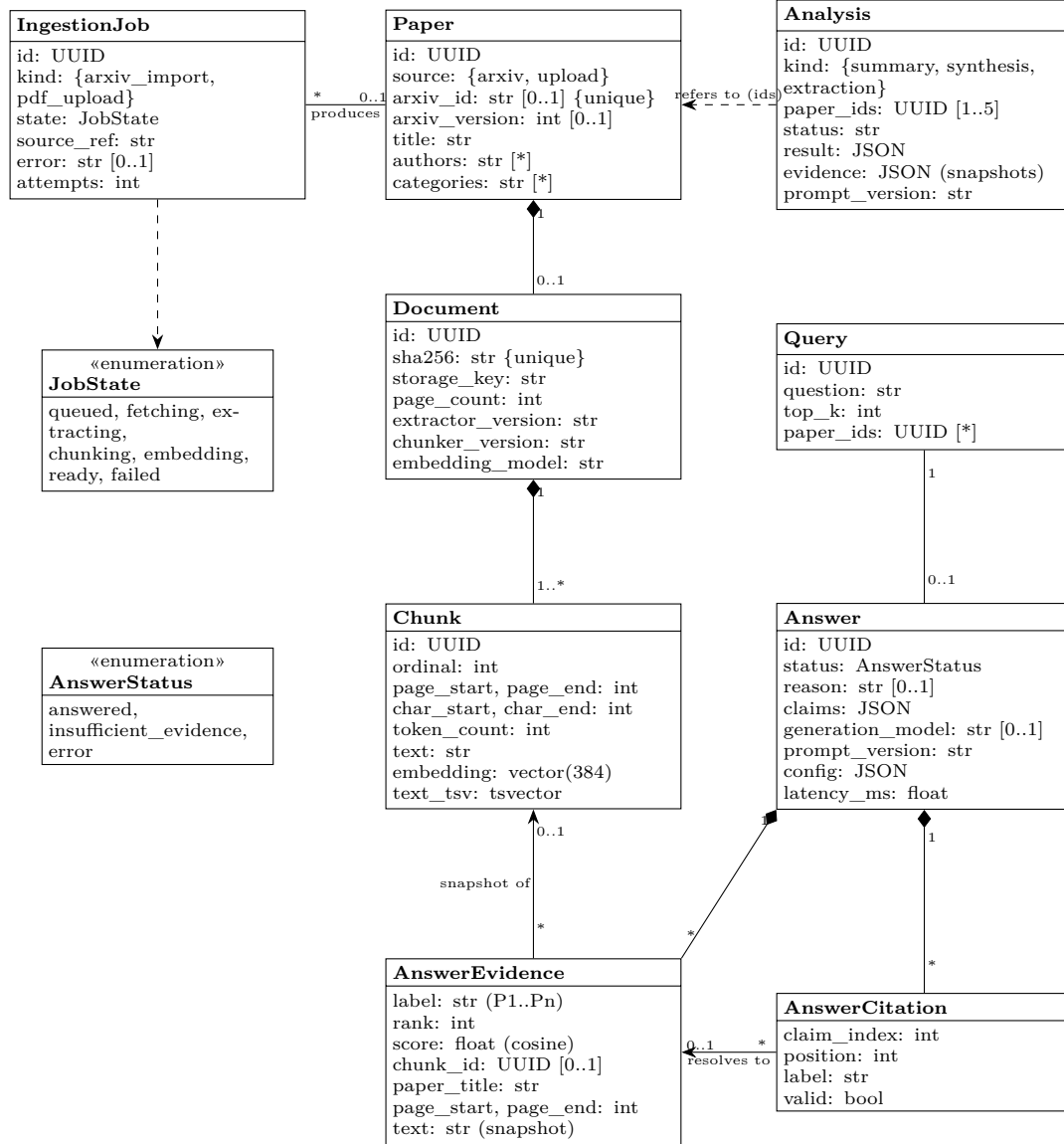


Figure 5.2: UML class diagram, part 1: persistent domain model

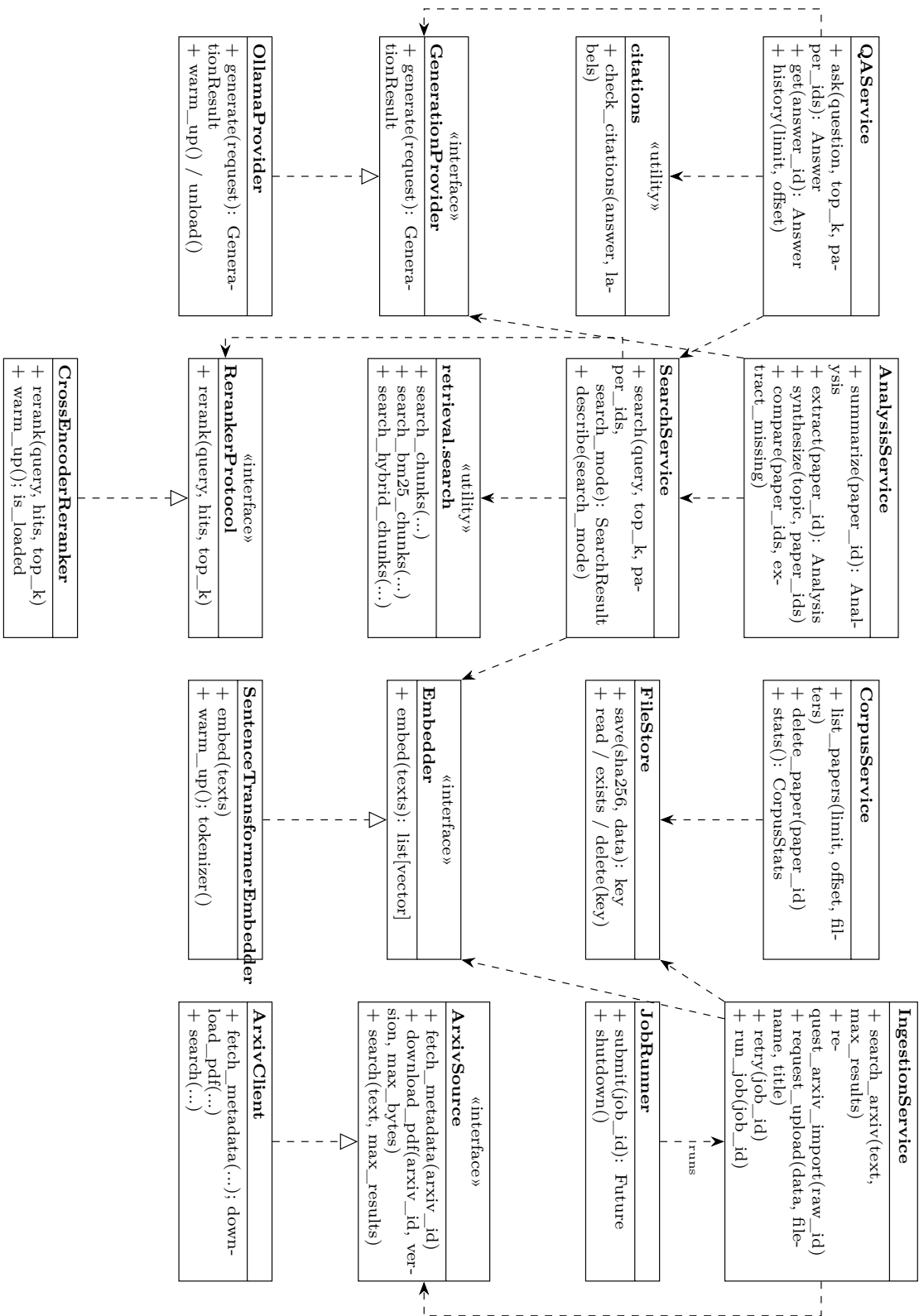


Figure 5.3: UML class diagram, part 2: services and abstractions

5.3 Sequence Diagram

Figure 5.4 follows a question from the interface to the answer and back, matching `QAService.ask`. After validation the service retrieves candidates (embedding, dense and lexical SQL, rank fusion and cross-encoder re-ranking inside `SearchService`), keeps passages whose cosine similarity is at least 0.30 and labels them. The outer `alt` fragment shows the early abstention without a model call. In the other operand the model is called; the inner `alt` fragment separates provider failures (Ollama unreachable, a timeout, or output that does not validate against the schema), which are persisted with status `error` and returned as HTTP 503, 504 or 502, from the normal path, in which citations are checked and the outcome is persisted as `answered` or `insufficient_evidence`. The diagram ends with the user opening a citation, which the interface resolves from the evidence already returned; no further request is made.

5.4 Activity Diagram

Figure 5.5 models ingestion in two swimlanes. The request lane is synchronous: an arXiv identifier is parsed, or an uploaded file is read with a size limit, checked (PDF signature, encryption, page limit) and hashed. Invalid requests return 422 and duplicate uploads 409 before any job exists. Valid uploads are stored under their hash, a job is created (or the in-flight job for the same source is returned) and handed to the job runner, and the request returns 202 at the fork. The background lane follows `run_job`: an arXiv job fetches metadata and finishes immediately if that version is already indexed with the current chunker; otherwise the PDF is downloaded (or the stored upload read), extracted page by page, chunked, embedded and written in one transaction. Any error raised inside the interruptible region—for example a network failure, HTTP 429 from arXiv, a PDF without a text layer, or an embedding failure—ends the job as `failed` with its message, and a rejected upload’s file is deleted.

5.5 Communication Diagram

Figure 5.6 shows the same interaction as Figure 5.4 as a communication (collaboration) diagram. It emphasises which objects are linked: the route talks only to `QAService`, which is the single object that talks to the language model, the citation checker and the database for persistence, while `SearchService` encapsulates the embedder, the database queries and the re-ranker. Message numbers are nested to show that messages 1.1.1.1 to 1.1.1.3 happen during `search`, and guards mark the conditional messages: generation happens only when evidence passes the threshold, and citation checking only when generation produced schema-valid output.

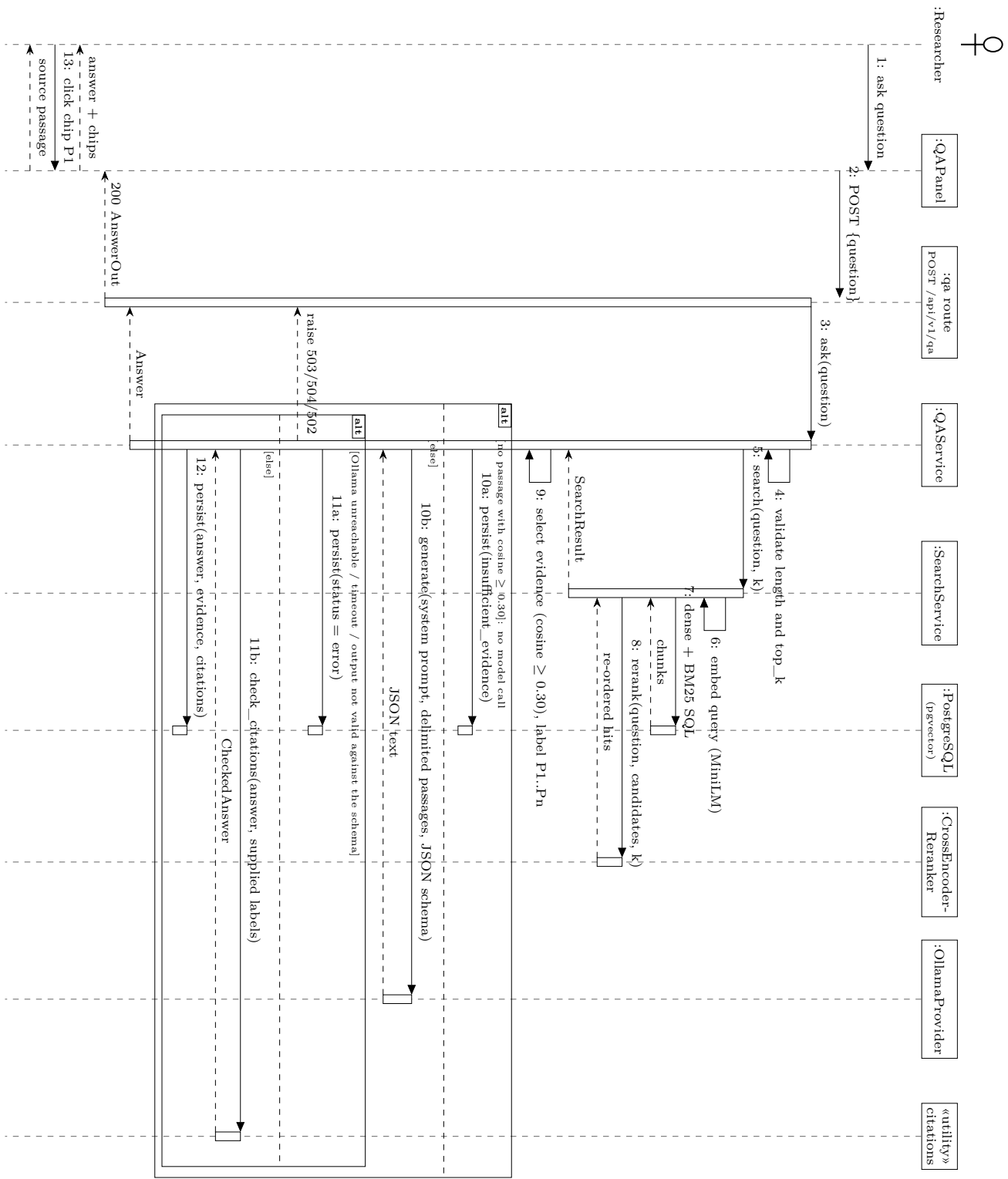


Figure 5.4: UML sequence diagram: grounded question answering

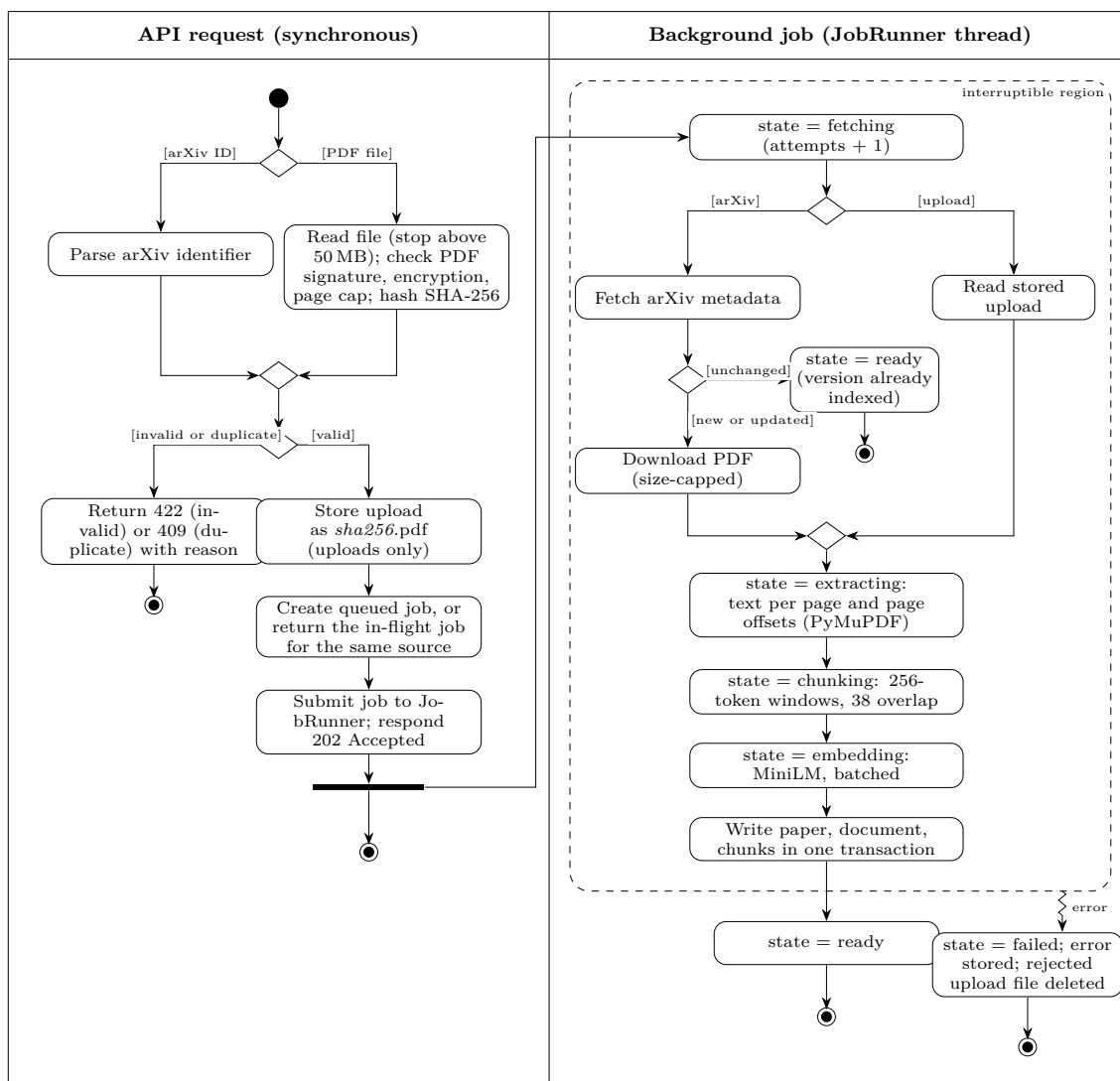


Figure 5.5: UML activity diagram: paper ingestion

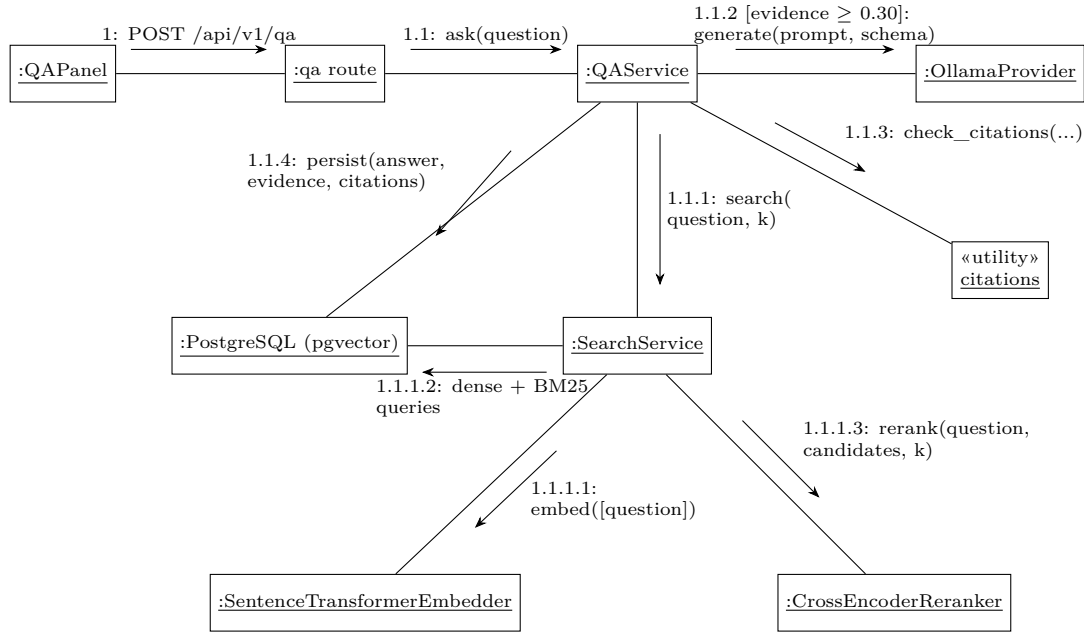


Figure 5.6: UML communication diagram: grounded question answering

5.6 Consistency Between the Diagrams

Table 5.1 cross-checks the dynamic diagrams against each other and against the code.

Table 5.1: Correspondence of sequence and communication messages

Sequence	Communication	Code
2, 3	1, 1.1	<code>api/v1/qa.py: ask → QAService.ask</code>
5	1.1.1	<code>SearchService.search</code>
6	1.1.1.1	<code>SentenceTransformerEmbedder.embed</code>
7	1.1.1.2	<code>search_hybrid_chunks (dense and BM25 SQL, RRF)</code>
8	1.1.1.3	<code>CrossEncoderReranker.rerank</code>
10b	1.1.2	<code>OllamaProvider.generate</code>
11b	1.1.3	<code>check_citations</code>
10a, 11a, 12	1.1.4	<code>QAService._persist</code>

Implementation

This chapter describes how each module is implemented. File paths are relative to the repository root. Code excerpts are shortened only by removing blank lines.

6.1 Paper Discovery and Metadata

`backend/app/ingestion/arxiv.py` implements the arXiv client. Identifiers are parsed by `ArxivId.parse`, which accepts old- and new-style identifiers with or without a version and rejects anything else with HTTP 422. Requests go only to an allow-list of HTTPS hosts, follow redirects only within that list, respect a minimum interval of three seconds between requests, and cap download size. Feeds are parsed with `defusedxml`. Keyword search keeps only word characters, dots and hyphens, so user input cannot inject arXiv query operators, and since the final acceptance run it drops common English stop words: arXiv does not index them, and because every term is combined with AND, a title such as “Attention Is All You Need” previously returned no results. Search results are marked with the version already in the corpus, if any.

6.2 PDF Upload and Parsing

Uploads are read in blocks and rejected as soon as they exceed 50 MB. Before any job is created, `inspect_pdf` checks the %PDF- signature, opens the file with PyMuPDF, and rejects corrupt, encrypted, empty and over-long (more than 200 pages) documents. The SHA-256 of the bytes detects duplicates (HTTP 409), and the file is stored as `<sha256>.pdf`; the user’s filename is used only as a display label. Extraction (`extract_pdf`) reads each page’s text, normalises it to Unicode NFC and trims trailing spaces, joins pages with a fixed separator, and records the character offset at which each page starts. `ExtractedDocument.page_at` maps any character offset back to its page by binary search. A document with fewer than 50 extracted characters per page on average is rejected as a probable scan, because optical character recognition is out of scope.

6.3 Chunking

`chunk_document` tokenises the extracted text with the embedding model’s own WordPiece tokenizer, so that chunk sizes match what the model sees. Windows of 256 to-

kens, including the two special tokens, advance by a stride of 216 tokens (an overlap of 38). Each chunk stores the exact substring of the document between its first and last token, its character span, and `page_start` and `page_end` computed from that span, so a chunk that crosses a page break records both pages. Chunk identifiers are name-based UUIDs derived from the document hash, the chunker version and the ordinal, so re-ingesting the same file with the same configuration reproduces them. The chunker configuration is stored as a version string (`tokwin-v1|...|w256|o38`).

6.4 Embedding Generation

`SentenceTransformerEmbedder` loads `all-MiniLM-L6-v2` on the CPU, keeping GPU memory for the language model. It embeds in batches, normalises vectors to unit length, and checks that every vector has 384 dimensions. The model is loaded by a warm-up thread at start-up; until it is loaded, the readiness endpoint reports `loading`.

6.5 Database Persistence

SQLAlchemy 2 models (`backend/app/db/models.py`) map the tables of chapter 4; the schema itself is created only by Alembic migrations 0001 to 0006. Migration 0006 replaced an expression index with a stored generated column, `text_tsv = to_tsvector('english', text)`, and a GIN index, because ranking recomputed the vector for every matching row: one query took 636ms before the change and 19ms after it on the evaluation corpus. All derived data of a document is written in a single transaction at the end of a job, so a failure never leaves partial chunks.

6.6 Semantic Retrieval

`SearchService.search` supports four modes. `dense` orders chunks by pgvector cosine distance. `bm25` ranks with `ts_rank_cd` over the OR of the query's alphanumeric terms. `hybrid` fuses the two candidate lists with reciprocal rank fusion, and `hybrid_rerank` (the default) re-orders the fused candidates with the cross-encoder. A hit always carries the cosine similarity as `score`, so the evidence threshold has one meaning in every mode, and the mode's own ordering score separately as `rank_score`. The fusion step is short:

Listing 6.1: Reciprocal rank fusion (`backend/app/retrieval/search.py`)

```
for hits, weight in ((dense_hits, dense_weight), (bm25_hits, sparse_weight)):
    for rank_idx, hit in enumerate(hits, start=1):
        cid = hit.chunk.id
        hit_map.setdefault(cid, hit)
```

```

        rrf_scores[cid] = rrf_scores.get(cid, 0.0) + weight / (rrf_k + rank_idx)
    ordered = sorted(rrf_scores, key=lambda cid: (-rrf_scores[cid], str(cid)))
    return [replace(hit_map[cid], rank_score=rrf_scores[cid]) for cid in ordered[:
        top_k]]

```

Ties are broken by chunk identifier, which keeps rankings deterministic. Each list contributes $\max(4k, 20)$ candidates for a final top k .

6.7 Context Assembly and Ollama Inference

QAService keeps retrieved passages with cosine similarity of at least 0.30 (the threshold calibrated in the evaluation milestone), up to 12,000 characters, and labels them P1 to Pn in rank order. The prompt (`generation/prompts.py`, version `qa-v1`) wraps every passage in a `<passage>` element with its paper title and pages. Any text inside a passage that could open or close such an element is escaped, and the system prompt says that passages are quoted source material whose instructions must be ignored. The request to Ollama carries a JSON schema that constrains the output to at most five claims, each listing up to six labels that match `P[0-9]{1,2}` before its text of at most 300 characters, followed by a status. Generation uses temperature 0, a fixed seed, a 4,096-token context and at most 512 new tokens. The adapter (`OllamaProvider`) maps a refused connection or a missing model to HTTP 503 and a timeout to 504; output that does not validate becomes HTTP 502.

6.8 Citation Validation and Source Navigation

Listing 6.2 shows the core of citation checking. Labels are normalised and de-duplicated, and a citation is valid only if its label was supplied to the model. A claim whose text consists only of labels is dropped. A claim is `cited` if at least one of its citations is valid; otherwise it is `unsupported`, and if no claim is cited the whole answer is withheld as insufficient evidence.

Listing 6.2: Citation checking (`backend/app/generation/citations.py`)

```

for raw in claim.citations:
    label = _normalize_label(raw)
    if not label or label in seen:
        continue
    seen.add(label)
    citations.append(CheckedCitation(label=label, valid=label in supplied_labels)
)
support: Support = "cited" if any(c.valid for c in citations) else "unsupported"

```

Every outcome—answered, insufficient evidence or error—is persisted with snapshots of the evidence, the citations (valid ones linked to their evidence row), the

model tag, prompt version, retrieval configuration and timings. In the interface, each citation is a chip; selecting it opens an inspector with the passage text, paper, pages, retrieval rank and score. Invalid citations are marked as such and never linked, and the interface states that semantic support is not verified automatically.

6.9 Summaries, Synthesis, Extraction and Comparison

`AnalysisService` gathers evidence inside the selected papers with several focused queries (for example about the problem, method and results), then applies the same evidence discipline with prompt version `analysis-v1`. A summary produces three to five cited claims for one paper; a synthesis produces cited claims across two to five papers on a topic; an extraction returns the fields task, method, dataset, metric, result and limitations, each either cited or `unknown`; a comparison builds a table from extractions and adds a caveat when the papers report on different datasets. It never ranks the papers.

6.10 Frontend Workflows

The React application (`frontend/src/features/`) provides a status bar that polls `/api/v1/ready`, and views for the dashboard, asking, passage search, the corpus (filters, details, two-step delete), comparison, adding papers (arXiv search, import by identifier, upload, recent jobs with retry), history and settings. All requests go through one typed client; errors are displayed from the response envelope. Component tests use Testing Library queries by role and label.

6.11 Configuration and Error Handling

All settings are read from `SLIP_`-prefixed environment variables with validated ranges (`backend/app/core/config.py`); Appendix B lists the most important ones. Domain errors are typed classes with a status code and an error code; a handler converts them into the error envelope. Database connection failures return 503 with an actionable message, and any other unexpected exception returns a fixed message while the details go to the server log.

Testing and Evaluation

All results in this chapter come from commands executed on the reference laptop (Table 3.5) between 8 and 9 October 2026, or from the project’s continuous-integration runs. Evaluation numbers are taken from run records committed under `eval/runs/`, which store the git commit, hardware, power source, dataset hash and full configuration. Where a measurement has a caveat, the caveat is stated with it.

7.1 Test Strategy

Tests are organised in layers, selected with pytest markers so that each layer runs where its dependencies exist. Tests that replace a component with a fake say so in their module documentation.

Table 7.1: Test layers and results in the final acceptance run (9 October 2026)

Layer	What is real / what is faked	Runs in CI	Passed
Backend unit	Pure code; fake embedder, scripted fake model, fake arXiv	Yes	140
Integration and model	Real PostgreSQL + pgvector and real MiniLM; fake model	Yes	83
Real local model	Ollama <code>qwen2.5:3b</code>	No	3
Live arXiv	arXiv API	No	1
Stack end-to-end	Running Docker stack through nginx, all real components	No	7
Frontend	Vitest and Testing Library with a mocked HTTP client	Yes	24

Static checks (Ruff lint and format, mypy in strict mode over 78 files, oxlint, the TypeScript compiler) passed, and the production frontend build succeeded. The live arXiv test failed once while arXiv was returning HTTP 429 to the machine and passed when re-run; it is reported as such. No test was skipped in the integration layer: in CI a missing test database makes the run fail rather than skip.

7.1.1 Unit, API and Integration Tests

Unit tests cover parsing of arXiv identifiers and feeds, query sanitisation, PDF extraction and page mapping, deterministic chunking, configuration validation, prompt

construction, the output schema, citation checking (invalid, duplicate and label-only citations), error mapping of the Ollama adapter, the evaluation metrics, search mode routing and fusion, and the error envelope. API tests use FastAPI's test client to check input bounds, pagination limits, CORS, request identifiers and readiness. Integration tests run the real pipeline against PostgreSQL: arXiv import (with a fake arXiv source), idempotent re-import, upload validation, duplicate detection, search in all four modes, question answering with a scripted model (including fabricated labels, abstention and generation failures), analyses, corpus management, the evaluation harness, and the provenance checker.

7.1.2 Database and Migration Tests

Integration tests apply all migrations to an empty database, run `alembic check` to detect drift between the ORM models and the migrations, perform a downgrade and upgrade round trip, and verify constraints such as cascading deletes and the vector dimension. A reproducibility check on a fresh clone migrated a new database from revision 0001 to 0006 on first start.

7.1.3 End-to-End Tests

The end-to-end suite runs against the Docker stack: it checks readiness, uploads a synthetic paper with a unique marker word, waits for ingestion, finds the paper with all four search modes, asks a question and checks that a citation is reported valid exactly when its label is among the evidence, requests a summary, and deletes the paper. During final acceptance the same journeys were also exercised manually with real papers (section 7.10).

7.2 Evaluation Data Set

- **Corpus:** 20 natural-language-processing papers pinned to exact arXiv versions (`eval/corpus.json`); 399 pages and 1,654 chunks.
- **Questions:** `eval/qa_v1.json`, 50 items: 40 answerable with 110 verbatim evidence quotes, and 10 unanswerable, several of them adversarial (they name a corpus paper and ask for a fact found only in a neighbouring paper).
- **Provenance of labels:** all items were written by the AI assistant used by the team (ADR-0007) by keyword search of the papers' text, not with the retriever. **No item has been reviewed by a person.** Each quote is verified to occur in the corpus before every run.
- **Relevance unit:** a retrieved chunk is relevant if it belongs to the labelled paper and contains one of the item's evidence quotes.

7.3 Metrics

For Q answerable questions, let r_q be the rank of the first relevant chunk for question q (undefined if none is in the top 10). Then

$$\text{Recall@}k = \frac{1}{Q} \sum_{q=1}^Q \mathbf{1}[r_q \leq k], \quad \text{MRR} = \frac{1}{Q} \sum_{q=1}^Q \frac{1}{r_q}, \quad (7.1)$$

with $1/r_q = 0$ when no relevant chunk is retrieved. $\text{Recall@}k$ is thus a hit rate, as in the proposal. Both are implemented in `backend/app/evaluation/metrics.py` and tested on hand-computed cases. Citation validity is the number of valid citations divided by all citations in answers returned as answered. Abstention recall is the fraction of unanswerable questions that receive “insufficient evidence”; abstention precision is the fraction of “insufficient evidence” outcomes that occur on unanswerable questions. Latency percentiles use the nearest-rank method.

7.4 Retrieval Results

Table 7.2 compares the four search modes. Configuration: top 10, RRF $k = 60$, candidate pool $\max(4k, 20)$ per list, AC power, CPU inference.

Table 7.2: Retrieval quality and search latency by mode (40 answerable questions)

Mode	R@1	R@5	R@10	MRR	p50 / p95 (ms)
dense (baseline)	0.200	0.700	0.775	0.416	42 / 49
bm25	0.300	0.525	0.650	0.393	56 / 91
hybrid (RRF)	0.375	0.725	0.825	0.527	102 / 137
hybrid_rerank (default)	0.575	0.875	0.925	0.701	1,734 / 1,955

Run records: dense and hybrid_rerank 20261008T214635Z and 20261008T214832Z (commit 03fb705); bm25 and hybrid 20261008T203750Z and 20261008T203825Z (commit 192ac2b).

Only `hybrid_rerank` meets the target $\text{Recall@5} \geq 0.80$ (NFR-01). Three caveats apply: the mode was chosen on the same 50 questions it is measured on; the labels are unreviewed; and with 40 questions the 95% Wilson interval for 35/40 is roughly 0.74 to 0.95, which includes values below the target. Every miss in every mode retrieved the correct paper but ranked the answering passage too low. Metrics were identical across four commits, which confirms that retrieval is deterministic.

7.5 Retrieval Latency Profile

Table 7.3 breaks retrieval into stages for the question-answering configuration (top 6, 24 candidates), measured with the project’s `profile-retrieval` command over the 50 questions.

Table 7.3: Retrieval stage latency at top 6 (ms, p50 / p95)

Stage	Latency
Query embedding (MiniLM, CPU)	17 / 30
Dense SQL (exact scan)	29 / 36
Lexical SQL	36 / 66
Hybrid call (dense + lexical + RRF)	61 / 85
Cross-encoder re-ranking (24 pairs, CPU)	895 / 985

The cross-encoder accounts for about 92% of retrieval time. The same profile found 47 pairs of neighbouring (overlapping) chunks among the 300 top-6 slots and no exact duplicates. At this corpus size the PostgreSQL planner uses an exact sequential scan (12 ms) instead of the HNSW index, so results are exact.

7.6 Question-Answering Results

Two complete runs over all 50 questions used the default retrieval, `qwen2.5:3b` at temperature 0, top 6 and the 0.30 threshold (Table 7.4). The first run also scored each claim with a local judge.

The false answer is the same in both runs: asked how much energy training DistilBERT consumed, the model answered with a figure from the LLaMA paper, citing a real passage about the wrong model. Run-to-run differences in the answer rate show that generation is not bit-reproducible even at temperature 0; results are therefore reported per run. Stored answers from run 2 give a median of 1.16 s for retrieval and 1.95 s for generation per question.

7.7 Citation Integrity

Citation integrity was checked deterministically, independently of the model:

- across all 418 answers stored in the evaluation database, every one of 422 valid citations resolves to an evidence row of the same answer, and no invalid citation is linked to evidence;

Table 7.4: Grounded question answering over the evaluation set

Measure	Run 1 (204611Z)	Run 2 (205117Z)
Answer rate on answerable questions	0.875	0.950
Answerable questions with a valid citation to a relevant passage	0.725	0.750
Citation validity (valid / all)	52 / 52	57 / 57
Unanswerable questions declined (abstention recall)	9 / 10	9 / 10
Abstention precision	9 / 14	9 / 11
False answers on unanswerable questions	1 / 10	1 / 10
Warm latency p50 / p95 (s)	3.10 / 4.50	3.06 / 4.43
Cold start (three samples, s)	8.4–12.0	8.1–10.2

- all 2,364 evidence snapshots equal the text, pages and paper of the chunk they came from;
- the provenance checker re-extracted all 20 PDFs and confirmed for all 1,654 chunks that the stored text equals the document text at the recorded span, that the recorded pages are correct, and that the chunk’s first and last words occur on those pages of the PDF (0 problems; 433 chunks span two or more pages).

Unit and integration tests cover non-existent labels, duplicate labels, label-only claims, malformed model output and deleted sources (evidence snapshots are kept with an empty chunk reference). Because the model can only emit labels, a fabricated page number or chunk identifier cannot enter the system.

7.8 Groundedness

A valid citation shows that a claim refers to a passage the model was given, not that the passage supports the claim. Two complementary checks were made:

- **Local judge** (judge-v1, the same qwen2.5:3b model): of 37 claims in run 1, 31 (83.8%) were rated supported, 3 partially supported and 3 not supported. The judge is not independent of the answering model.
- **Spot check by the AI assistant**, not by a person: 14 claims (the 6 the judge flagged and 8 random claims it rated supported) were read against the full cited text. Ten were supported and answered the question; one was supported but did not answer it; one was a wrong-paper answer (a Transformer question

answered with LLaMA’s optimiser settings); two were not stated in the passage they cited. The judge agreed with this reading on 11 of 14 claims.

No human spot check has been performed; it is required by the project’s evaluation rules and remains open.

7.9 Abstention, Failure Handling and Prompt Injection

The system abstains at three points: when no passage reaches the threshold (no model call), when the model reports insufficient evidence, and when no claim has a valid citation. In the evaluation runs all abstentions on unanswerable questions came from the model. Failure handling was tested live: with the database stopped, readiness reports the cause and the API returns 503 with an actionable message, recovering when the database restarts; with Ollama unreachable, question answering returns 503 while search keeps working; invalid input returns 422 with the offending field. A real-model test placed an instruction inside a retrieved passage; the model did not follow it (`test_instructions_inside_passages_are_not_followed`).

7.10 Final Acceptance Journeys

On 9 October 2026 the rebuilt stack was exercised with real components: discovering papers through arXiv search; importing 2004.04906v3 (and confirming that a second import created no duplicate); uploading a real five-page paper; re-checking provenance for all four stored documents (210 chunks, 0 problems); confirming 384-dimensional embeddings for all chunks; searching; asking “How much smaller and faster than BERT is DistilBERT?” (answered “40% smaller and 60% faster” with two citations whose passages contain both figures, one spanning pages 4–5); asking an unsupported question (declined before any model call); running summary, extraction, synthesis and comparison; filtering and deleting papers; and opening citations in the user interface. One defect found during this run (arXiv title search with stop words returned nothing) was fixed and covered by a regression test.

7.11 Continuous Integration

GitHub Actions runs six jobs on every push: backend lint, types and unit tests; integration tests against a pgvector service container with the real embedding model; frontend checks and build; Docker image build; a retrieval regression gate that rebuilds the evaluation corpus from arXiv and fails if Recall@5 drops below 0.85 for

`hybrid_rerank` or 0.675 for `dense` (each one question below its baseline); and dependency scanning with `pip-audit` and `npm audit`. One run on the main branch failed because arXiv rate-limited the runner; the corpus builder now retries failed downloads with back-off. CI does not use Ollama, so real-model and end-to-end tests are run manually.

Security, Privacy and Reliability

The system was reviewed twice for security: a first pass during development and a second pass before the MVP submission (`docs/security/mvp-security-audit.md`). Both were practical reviews by the AI assistant used by the team—code reading, the test suites, targeted probes against the running stack and a check of the host’s open ports. **They were not penetration tests or a comprehensive security audit.** The threat model is a single user on one machine with all application ports bound to `127.0.0.1`.

8.1 Implemented Controls

8.2 Untrusted Retrieved Text and Model Output

Retrieved passages are treated as untrusted data (chapter 2). The model output is also untrusted: it must validate against a schema, its citations are resolved against the supplied passages, and a failure produces an explicit error rather than an answer. A structurally valid citation is labelled *cited*, not *supported*, and the interface states that semantic support is not verified.

8.3 Local Model Deployment

Generation runs only on the user’s machine through Ollama; there is no hosted fallback. During the second review, the host’s Ollama service was found listening on all network interfaces, with inbound firewall rules allowing access on a network classified as public. This exposes Ollama’s unauthenticated API to the local network. It is a host setting rather than an application defect; the remediation (binding Ollama to `127.0.0.1` and disabling the inbound rules) is documented and must be applied by the machine owner. It was not applied during the review.

8.4 Logging and Retention

Logs are JSON lines with request identifiers. The web server’s access log, which recorded query strings, was disabled. The database retains papers, chunks and the full question history, including evidence snapshots, until the user deletes them; there is no automatic retention limit.

Table 8.1: Security and privacy controls and how they were verified

Area	Control	Verification
Upload validation	50 MB cap (nginx 55 MB), PDF signature, full open, encryption and page checks before any job	Tests; live probes: non-PDF, truncated and 51 MB files 422, 60 MB 413
Storage	Server-generated <sha256>.pdf names; keys validated to stay inside the storage root	Test of unsafe keys; upload named ../../etc/passwd.pdf stored inside storage
Database	ORM and bound parameters only; lexical query terms reduced to word characters	Code search; injection-style queries left tables intact
Outbound requests	arXiv hosts allow-listed (HTTPS), redirects re-checked, size caps, defusedxml	Tests
Prompt injection	Delimited, neutralised passages; instructions to ignore passage instructions; JSON schema	Real-model test
Citation integrity	Labels only; server-side resolution; answers without valid citations withheld	Tests; database-wide checks (chapter 7)
Errors	Typed errors to status codes; fixed message for unexpected errors; 503 for database outage	Tests; live probes
Resource bounds	Limits on query and question length, top- k , pagination, context, tokens, timeouts, one ingestion worker	Tests
CORS and headers	Origin allow-list; CSP, nosniff, frame denial, no referrer	Test; curl -I
Secrets	.env ignored by git; only .env.example committed; settings endpoint without credentials	Git history scan; test
Privacy	No hosted model; logs contain event names, identifiers and counts, not questions or document text	Log review
Dependencies	pip-audit and npm audit in CI	No known vulnerabilities at the time of review

8.5 Availability and Recovery

Readiness reports each dependency separately; search continues without Ollama. Jobs interrupted by a restart are marked failed on start-up and can be retried. Availability was measured only over a 25.5-minute window (52 of 52 probes successful), which is too short to support the 99% target. Backups use `pg_dump` together with a copy of the storage volume (`docs/operations.md`).

8.6 Open Risks

- No authentication or authorisation: acceptable only while the system stays single-user and local.
- The application connects to PostgreSQL as a superuser; a least-privilege runtime role is planned.
- PDF parsing has no wall-clock timeout (bounded by the size and page limits and a single worker).
- Model revisions are pinned by name, not by revision, and are downloaded on first start.
- The CPU build of PyTorch is not covered by `pip-audit`.

Results, Limitations and Future Work

9.1 Demonstrated Capabilities

The final acceptance run concluded that the MVP is *ready with documented limitations*: every mandatory user journey works against the running application with real components, and no release-blocking defect remains. The demonstrated capabilities are arXiv discovery and idempotent import, validated PDF upload, page-aware extraction with verified provenance, semantic search in four modes, grounded question answering with server-verified citations and explicit abstention, summaries, syntheses, structured extractions and comparison tables, corpus management, question history, a dashboard and a settings view, and graceful handling of invalid input and missing services.

9.2 Requirements Achieved

Table 9.1 summarises the status of the requirements; Appendix C gives the per-requirement detail.

Table 9.1: Requirement status at final acceptance

Status	Requirements
Verified	FR-01–FR-10, FR-12–FR-18, FR-21, FR-22, FR-24, FR-25; IR-01–IR-09; NFR-01 (on this set, with caveats), NFR-02, NFR-04, NFR-06–NFR-09, NFR-11, NFR-13, NFR-14, NFR-16
Partially implemented	FR-11 (one false answer in ten; no “uncertain” status), FR-19 (labels not reviewed by a person), FR-20 (no human spot check), FR-23 (settings omit the search mode), NFR-05 (pilot too short), NFR-10 and NFR-12 (open security items)
Implemented but unverified	NFR-15 (no manual accessibility review)
Failed	NFR-03 (95th-percentile latency 4.4–4.5 s against 3 s)
Deferred	FR-26 (optical character recognition)

9.3 Known Limitations

- **Latency.** The cross-encoder (about 0.9 s on the CPU) and generation (about 1.9 s at the median) keep the 95th percentile above 3 s. The faster **hybrid** mode saves about 1.1 s per question but lowers Recall@5 to 0.725.
- **Faithfulness.** The model can answer from a related paper (the wrong-paper failure) or cite a passage that does not state the claim. Structural citation checks cannot detect either.
- **Abstention on answerable questions.** The system sometimes declines questions that the corpus can answer, either because the answering passage is not retrieved or because the model abstains.
- **Evaluation evidence.** The 50 labels were written by an AI assistant and not reviewed by a person; the default retrieval mode was selected on the same questions; the set has almost no multi-paper questions and none with conflicting or ambiguous evidence.
- **Extraction from tables.** Plain-text extraction flattens tables, so a cited value can still be wrong (one perplexity value was reported as a BLEU score in a manual test).
- **Scope.** No authentication, no OCR, no metadata for uploaded papers, single machine only.

9.4 Technical Debt

The PostgreSQL role used by the application is a superuser; PDF parsing has no timeout; model revisions are not pinned; the HNSW index is unused at the current corpus size and its tuning parameters have no effect; neighbouring overlapping chunks are not de-duplicated before prompting; and the planned small-to-big chunking experiment (ADR-0003, option 3) was not run.

9.5 Future Work

The following are proposed, not implemented:

1. Human review of a sample of the evaluation labels, and a new held-out question set covering multi-paper, conflicting, ambiguous and partially supported cases, so that further tuning does not overfit.

2. A claim-to-passage entailment check and a check that the cited paper matches the paper named in the question, evaluated on the held-out set; an “uncertain” answer status.
3. Latency work: a smaller re-ranking pool, re-ranking on the GPU, or a smaller generation budget, each measured against both recall and latency.
4. Table-aware extraction and OCR for scanned papers.
5. A least-privilege database role, a parsing timeout, pinned model revisions, and authentication before any shared deployment.
6. A longer availability pilot with the existing probe script.

9.6 Maintenance and Extension at the Institute

The repository is organised for handover: requirements with permanent identifiers, a traceability matrix, eight decision records, a progress log of every milestone, setup and operations guides, audit reports, and automated tests with a CI pipeline. New retrieval or generation components can be added behind the existing interfaces (`Embedder`, `RerankerProtocol`, `GenerationProvider`) and evaluated with the same harness and regression gate. A different local model can be selected with one setting, and is recorded with every answer.

Conclusion

The project set out to let a user ask questions over a collection of scientific papers and receive answers whose every claim can be traced to a passage and a page, with an explicit refusal when the papers do not support an answer. The resulting system ingests papers from arXiv and from uploaded PDFs, preserves provenance from page to chunk, retrieves passages by combining dense and lexical search with cross-encoder re-ranking, and generates answers with a local model whose citations are resolved on the server.

The principal engineering decisions were to keep all inference local and refuse any hosted fallback, to let the model cite only passage labels so that the server can verify every citation, to persist evidence snapshots so that history survives changes to the corpus, and to treat evaluation as part of the system: a versioned evaluation set, reproducible run records, and a regression gate in continuous integration.

The verification evidence is mixed in a way the report has tried to state precisely. Citation integrity and provenance were verified deterministically across every stored answer and every stored chunk, and the default retrieval configuration reached a Recall@5 of 0.875 on the evaluation set. Against that, the latency target was not met, one in ten unanswerable questions received a wrong-paper answer, spot checks found claims not supported by their cited passage, and the evaluation labels have not yet been reviewed by a person.

For the course, the project exercised the full software-engineering cycle: requirements with stable identifiers, architecture decisions recorded with their alternatives, design modelled in UML, layered automated testing, security and reproducibility reviews, and an acceptance decision based on evidence rather than a successful build. For the institute, it provides a documented, locally deployable base that future students can extend and evaluate with the same tools.

References

- [1] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, et al. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38. DOI: [10.1145/3571730](https://doi.org/10.1145/3571730).
- [2] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. 2020. arXiv: [2005.11401](https://arxiv.org/abs/2005.11401).
- [3] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2023. arXiv: [2312.10997](https://arxiv.org/abs/2312.10997).
- [4] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2020, pp. 6769–6781. DOI: [10.18653/v1/2020.emnlp-main.550](https://doi.org/10.18653/v1/2020.emnlp-main.550).
- [5] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3980–3990. DOI: [10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410).
- [6] Stephen Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Foundations and Trends in Information Retrieval* (2009). DOI: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- [7] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2009, pp. 758–759. DOI: [10.1145/1571941.1572114](https://doi.org/10.1145/1571941.1572114).
- [8] Rodrigo Nogueira and Kyunghyun Cho. *Passage Re-ranking with BERT*. 2019. arXiv: [1901.04085](https://arxiv.org/abs/1901.04085).
- [9] Raghav B. Venkataramaiyer. *tietreport: TIET University Report LaTeX Class, version 1.0.4*. URL: <https://ctan.org/pkg/tietreport> (visited on 10/09/2026).
- [10] arXiv. *arXiv API User’s Manual*. URL: <https://info.arxiv.org/help/api/user-manual.html> (visited on 10/09/2026).
- [11] PyMuPDF developers. *PyMuPDF Documentation*. URL: <https://pymupdf.readthedocs.io/en/latest/> (visited on 10/09/2026).

- [12] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. 2017. arXiv: [1706.03762](#).
- [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: [10.18653/v1/N19-1423](#).
- [14] Sentence Transformers. *all-MiniLM-L6-v2 model card*. URL: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2> (visited on 10/09/2026).
- [15] Sentence Transformers. *Sentence Transformers Documentation*. URL: <https://www.sbert.net/> (visited on 10/09/2026).
- [16] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*. 2020. arXiv: [2002.10957](#).
- [17] Yu. A. Malkov and D. A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836. DOI: [10.1109/TPAMI.2018.2889473](#).
- [18] pgvector contributors. *pgvector: Open-source vector similarity search for Postgres*. URL: <https://github.com/pgvector/pgvector> (visited on 10/09/2026).
- [19] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. URL: <https://nlp.stanford.edu/IR-book/>.
- [20] The PostgreSQL Global Development Group. *PostgreSQL 17 Documentation: Full Text Search*. URL: <https://www.postgresql.org/docs/17/textsearch.html> (visited on 10/09/2026).
- [21] Sentence Transformers. *cross-encoder/ms-marco-MiniLM-L-6-v2 model card*. URL: <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2> (visited on 10/09/2026).
- [22] Nelson F. Liu et al. “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 157–173. DOI: [10.1162/tacl_a_00638](#).
- [23] Ellen M. Voorhees. *The TREC-8 Question Answering Track Report*. Proceedings of the Eighth Text REtrieval Conference (TREC-8), NIST. 1999. URL: https://trec.nist.gov/pubs/trec8/papers/qa_report.pdf.

- [24] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. 2023. arXiv: [2309.15217](#).
- [25] Lianmin Zheng et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. 2023. arXiv: [2306.05685](#).
- [26] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection”. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. ACM, 2023, pp. 79–90. DOI: [10.1145/3605764.3623985](#).
- [27] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. URL: <https://genai.owasp.org/llm-top-10/> (visited on 10/09/2026).
- [28] Ollama. *Ollama*. URL: <https://ollama.com/> (visited on 10/09/2026).
- [29] Qwen Team, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. *Qwen2.5 Technical Report*. 2024. arXiv: [2412.15115](#).
- [30] FastAPI developers. *FastAPI Documentation*. URL: <https://fastapi.tiangolo.com/> (visited on 10/09/2026).
- [31] SQLAlchemy authors and contributors. *SQLAlchemy and Alembic Documentation*. URL: <https://www.sqlalchemy.org/> (visited on 10/09/2026).
- [32] Meta Open Source. *React Documentation*. URL: <https://react.dev/> (visited on 10/09/2026).

Setup and Execution

These steps reproduce the setup that was verified on a fresh clone (docs/operations/mvp-setup-and-testing.md).

A.1 Prerequisites

Docker Desktop with Compose; Ollama running on the host with the model pulled (ollama pull qwen2.5:3b, about 2 GB); optionally uv and Node.js 24 for local development. Clone the repository outside OneDrive: Docker builds from a OneDrive folder failed on the reference machine.

A.2 Start the Stack

```
git clone https://github.com/diyajain0902/UCS503P-202627-Scientific-Literature-Intelligence-Platform-Project.git
cd UCS503P-202627-Scientific-Literature-Intelligence-Platform-Project
cp .env.example .env # set POSTGRES_PASSWORD; never commit .env
docker compose up -d --build
```

Database migrations run automatically when the backend starts (alembic upgrade head). Wait until <http://localhost:8080/api/v1/ready> reports ready with the database, embedding model, re-ranker and Ollama all available; the first start downloads two models of about 90 MB each (about 15 minutes on the reference network). Open <http://localhost:8080>. Stop with `docker compose down`; `docker compose down -v` deletes all data.

A.3 Run the Tests

```
cd backend
uv run ruff check . && uv run ruff format --check . && uv run mypy
uv run pytest -m "not integration and not model and not network and not ollama
and not e2e"
SLIP_TEST_DATABASE_URL=postgresql+psycopg://slip:<password>@127.0.0.1:5432/
slip_test \
  uv run pytest -m "integration or model"
uv run pytest -m ollama # needs Ollama
SLIP_E2E_BASE_URL=http://localhost:8080/api/v1 uv run pytest -m e2e # needs the
running stack
```

```
cd ../frontend && npm ci && npm run lint && npm run typecheck && npm test && npm run build
```

A.4 Run the Evaluation

With `SLIP_DATABASE_URL` pointing at a separate `slip_eval` database:

```
uv run alembic upgrade head
uv run python -m app.evaluation build-corpus
uv run python -m app.evaluation check-labels
uv run python -m app.evaluation retrieval --search-mode hybrid_rerank
uv run python -m app.evaluation qa # needs Ollama; add --judge for the local judge
```

A.5 Demonstration Sequence

1. Dashboard: the status bar shows the database, embedding model, re-ranker and Ollama as available.
2. Add papers: import 1706.03762v7 by identifier, or search arXiv and import a result; upload a PDF.
3. Search passages for a concept; note the pages and scores.
4. Ask “What BLEU score does the big Transformer model achieve on the WMT 2014 English-to-German translation task?” and open citation P1 to show the page-8 passage containing 28.4.
5. Ask a question the papers do not cover to show the insufficient-evidence response.
6. Summarise or extract a paper; compare two papers.
7. Reopen an answer from History; show the Settings view.

API and Configuration Reference

B.1 REST Endpoints

All endpoints are under `/api/v1`; the OpenAPI document is served at `/api/v1/openapi.json`.

Table B.1: REST endpoints

Method	Path	Purpose
GET	<code>/health</code>	Liveness
GET	<code>/ready</code>	Readiness per dependency
GET	<code>/arxiv/search?q=&max_results=</code>	Search arXiv
POST	<code>/papers/arxiv</code>	Import by arXiv identifier (202 with job)
POST	<code>/papers/upload</code>	Upload a PDF (multipart; 202 with job)
GET	<code>/jobs, /jobs/{job_id}</code>	List jobs, job status
POST	<code>/jobs/{job_id}/retry</code>	Retry a failed job
GET	<code>/papers, /papers/{paper_id}</code>	List with filters, paper details
DELETE	<code>/papers/{paper_id}</code>	Delete a paper, its chunks and file
GET	<code>/categories, /stats</code>	Corpus categories, dashboard statistics
POST	<code>/search</code>	Passage search (<code>query</code> , <code>top_k</code> , <code>paper_ids</code> , <code>search_mode</code>)
POST	<code>/qa</code>	Ask a question
GET	<code>/qa, /qa/{answer_id}</code>	History, one stored answer
POST	<code>/papers/{paper_id}/summary</code>	Summarise a paper
POST	<code>/papers/{paper_id}/extraction</code>	Structured extraction
GET	<code>/papers/{paper_id}/latest/{kind}</code>	Latest summary or extraction
POST	<code>/synthesis, /compare</code>	Cross-paper synthesis, comparison table
GET	<code>/analyses/{analysis_id}</code>	One stored analysis
GET	<code>/settings</code>	Read-only configuration (no credentials)

B.2 Main Configuration Settings

Settings are environment variables with the prefix `SLIP_` (`backend/app/core/config.py`).

Table B.2: Main configuration settings

Setting	Default	Meaning
<code>SLIP_OLLAMA_MODEL</code>	<code>qwen2.5:3b</code>	Local generation model
<code>SLIP_OLLAMA_TIMEOUT_SECONDS</code>	60	Generation timeout
<code>SLIP_OLLAMA_MAX_TOKENS</code>	512	Maximum generated tokens
<code>SLIP_SEARCH_MODE</code>	<code>hybrid_rerank</code>	<code>dense</code> , <code>bm25</code> , <code>hybrid</code> or <code>hybrid_rerank</code>
<code>SLIP_QA_DEFAULT_TOP_K</code>	6	Passages retrieved for question answering
<code>SLIP_QA_MIN_SCORE</code>	0.30	Minimum cosine similarity for evidence
<code>SLIP_QA_MAX_CONTEXT_CHARS</code>	12000	Evidence budget per question
<code>SLIP_MAX_PDF_BYTES</code>	50 MB	Upload size limit
<code>SLIP_MAX_PDF_PAGES</code>	200	Page limit
<code>SLIP_CHUNK_WINDOW_TOKENS</code>	256	Chunk window (overlap 38)
<code>SLIP_INGESTION_WORKERS</code>	1	Concurrent ingestion jobs (1–4)
<code>SLIP_CORS_ORIGINS</code>	<code>["http://localhost:5173"]</code>	Allowed browser origins

Requirements Traceability Matrix

Status as determined in the final acceptance run of 9 October 2026 (`docs/release/mvp-requirements-matrix.md`). V = verified, P = partially implemented, U = implemented but unverified, F = failed, D = deferred. Module paths are relative to `backend/app/` unless stated otherwise.

Table C.1: Requirements traceability

ID	Implementation	Verification evidence	St.
FR-01	<code>ingestion/arxiv.py</code>	Unit tests incl. stop-word regression; live search	V
FR-02	<code>services/ingestion.py</code>	Ingestion pipeline tests; live idempotent re-import	V
FR-03	<code>ingestion/pdf.py</code> , <code>api/v1/corpus.py</code>	Upload tests; live probes	V
FR-04	<code>ingestion/pdf.py</code>	PDF tests; provenance check, 0 problems	V
FR-05	<code>ingestion/chunking.py</code>	Chunking tests; provenance check	V
FR-06	<code>retrieval/embedding.py</code>	Real-model tests; 384-dimension check	V
FR-07	<code>db/</code> , migrations 0001–0006	Schema, drift and round-trip tests	V
FR-08	<code>retrieval/search.py</code> , <code>services/search.py</code>	Search integration tests (4 modes); live	V
FR-09	<code>services/qa.py</code> , <code>generation/</code>	Q&A tests; real-model tests; live answer	V
FR-10	<code>generation/citations.py</code>	Tests; 422/422 stored citations resolve	V
FR-11	<code>services/qa.py</code>	Tests; 9/10 abstentions; 1 false answer	P
FR-12	<code>services/analysis.py</code>	Analysis tests; live summary	V
FR-13	<code>services/analysis.py</code>	Analysis tests; live synthesis	V
FR-14	<code>services/analysis.py</code>	Analysis tests; live extraction	V
FR-15	<code>services/analysis.py</code>	Analysis tests; live comparison	V
FR-16	<code>services/corpus.py</code> ; frontend corpus view	Corpus tests; live filters and delete	V
FR-17	<code>services/ingestion.py</code> , <code>services/jobs.py</code>	Job and retry tests; live job states	V
FR-18	Q&A tables; frontend history view	Tests; live history	V

ID	Implementation	Verification evidence	St.
FR-19	evaluation/; eval/	Metric tests; reproducible runs; labels unreviewed	P
FR-20	evaluation/runner.py, groundedness.py	Runs with judge; no human spot check	P
FR-21	.github/workflows/ci.yml	Retrieval gate in CI	V
FR-22	/stats; dashboard view	Tests; live	V
FR-23	/settings; settings view	Tests; omits search mode	P
FR-24	retrieval/search.py	Tests; Recall@5 0.725	V
FR-25	retrieval/rerank.py	Tests; Recall@5 0.875	V
FR-26	–	Scanned PDFs rejected with a clear error	D
IR-01–09	API, config, Compose, CI, health, logging, Olama adapter, job runner, documentation	Tests, live probes, fresh-clone setup, CI	V
NFR-01	Retrieval	Recall@5 0.875 (caveats in chapter 7)	V
NFR-02	Retrieval	MRR 0.701	V
NFR-03	Q&A	p95 4.43–4.50 s against 3 s	F
NFR-04	Search	Search latency reported separately	V
NFR-05	Operations	52/52 probes in 25.5 minutes only	P
NFR-06	Citations	All displayed citations resolve	V
NFR-07	Abstention	Recall 0.90, precision 0.64–0.82 reported	V
NFR-08	Determinism	Identical metrics across commits	V
NFR-09	Run records	Model, prompt and retrieval configuration stored	V
NFR-10	Security	Second review passed with open items	P
NFR-11	Privacy	No content in logs; no hosted model	V

ID	Implementation	Verification evidence	St.
NFR-12	Bounds	No parse timeout; backend body size uncapped	P
NFR-13	Hardware	All runs on the reference laptop	V
NFR-14	CI	Green on the main branch	V
NFR-15	Accessibility	Role- and label-based tests only	U
NFR-16	Degradation	Live: search 200, Q&A 503 without Ol-lama	V

Evaluation Run Records

Every reported evaluation number comes from one of these run records in `eval/runs/`. Each record stores the git commit, whether tracked files were modified, hardware and power source, the dataset and corpus-manifest hashes, the full configuration and per-item results.

Table D.1: Run records cited in this report

Record	Commit	Content
20261008T214635Z_retrieval	03fb705	Dense retrieval, top 10
20261008T214832Z_retrieval	03fb705	Hybrid with re-ranking, top 10
20261008T214608Z_profile-retrieval	03fb705	Stage latency and redundancy, top 6
20261008T203750Z_retrieval	192ac2b	BM25-style lexical retrieval, top 10
20261008T203825Z_retrieval	192ac2b	Hybrid (RRF), top 10
20261008T204611Z_qa	b173e72	Q&A, hybrid with re-ranking, local judge
20261008T205117Z_qa	b173e72	Q&A, hybrid with re-ranking
20261008T170309Z_retrieval	3eab94d	Dense baseline (M3)
eval/uptime/20261009_pilot_window.jsonl		Availability probes (52)

D.1 Sample Output (real)

The following answer was produced by the running system during final acceptance (question, then the single claim and its validated citations), using the uploaded DistilBERT paper.

```
Question: How much smaller and faster than BERT is DistilBERT?
Status: answered (model qwen2.5:3b, prompt qa-v1, 3.4 s)
Claim 1: DistilBERT is 40% smaller and 60% faster than BERT.
         P1 valid -> DistilBERT (uploaded PDF), p. 4
         P3 valid -> DistilBERT (uploaded PDF), pp. 4-5
```